

Machine Learning

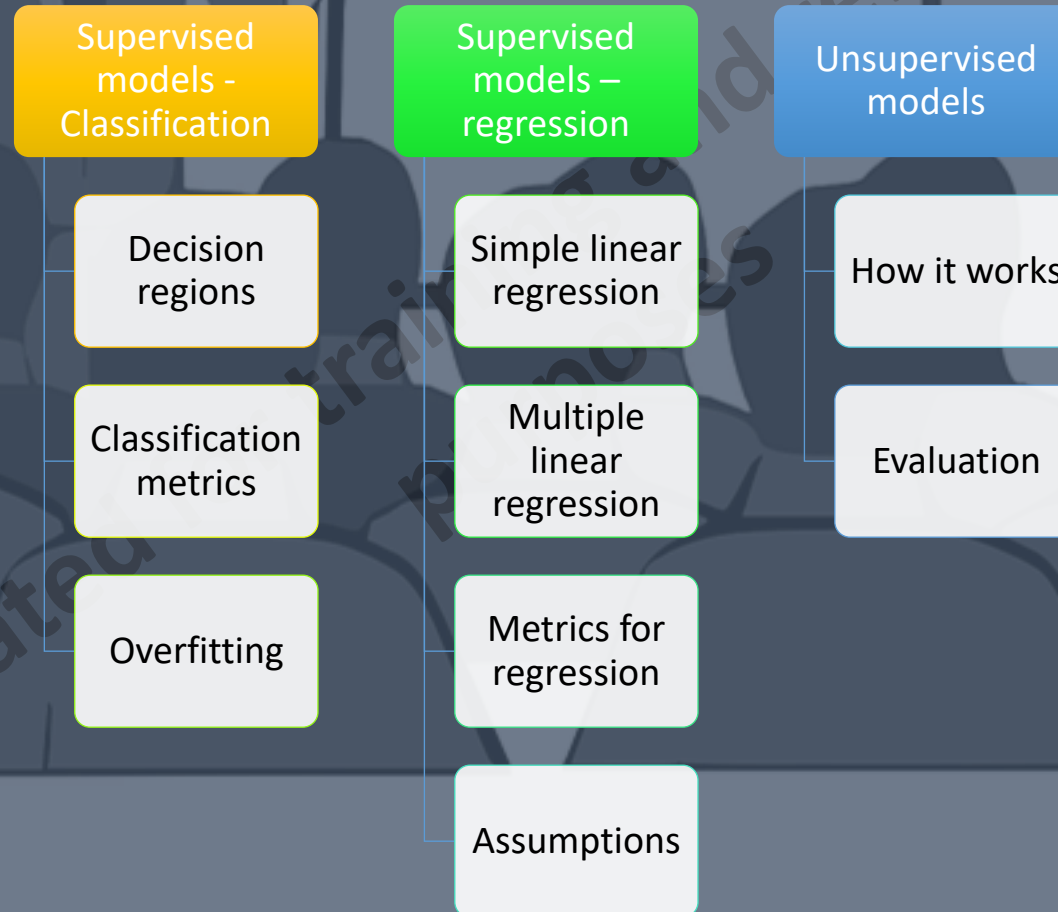
Supervised &
Unsupervised models
workflow

Step by step - intuitions



GROKKERS
AI FOR EVERYONE

Topics



Pre-reqs



Python, NumPy, pandas, plots



Data processing steps



ML – overview, examples

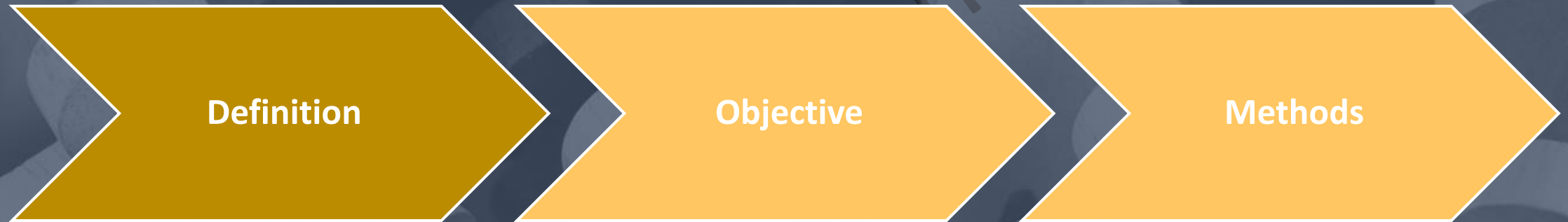


Supervised models

Decision Regions,
Metrics (accuracy, recall, confusion matrix)
Overfitting / generalization
Training data - sufficiency



Problem statement – Customer Segmentation



- process of dividing a customer base into distinct groups or segments based on shared characteristics, behaviors, or demographics.
- helps businesses tailor their marketing, product offerings, and services to better meet the specific requirements of each segment.
- Demographic Segmentation
- Behavioral Segmentation
- Geographic Segmentation

Data setup

Introduction:

- four customer classes based on income and spend predictors.

Predictor Columns:

- Column 1: Income
- Column 2: Spend

Income and Spend Ranges:

- Income: 1000 to 100000
- Spend: 1000 to 100000

Customer Classes:

- ES (Economically Stressed): Low income, Low spend
- SC (Savings Customer): High income, Low spend
- RC (Risk Customer): Low income, High spend
- PC (Prudent Customer): High income, High spend



Machine learning – Input and output



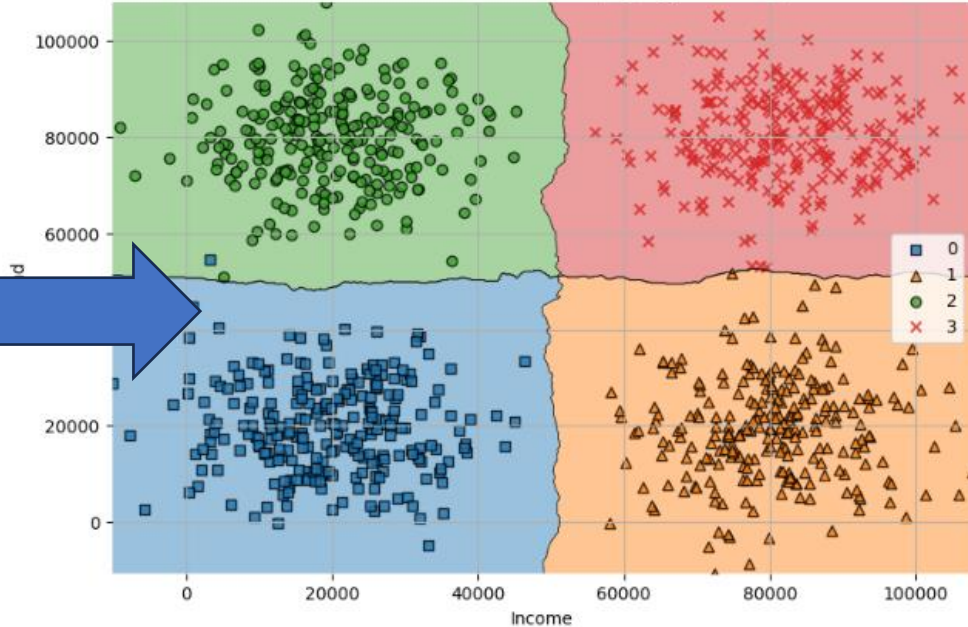
Input Data :

Income	Spend	Customer Type



Learning from Data

Map {Income, Spend} to
{Customer Type}



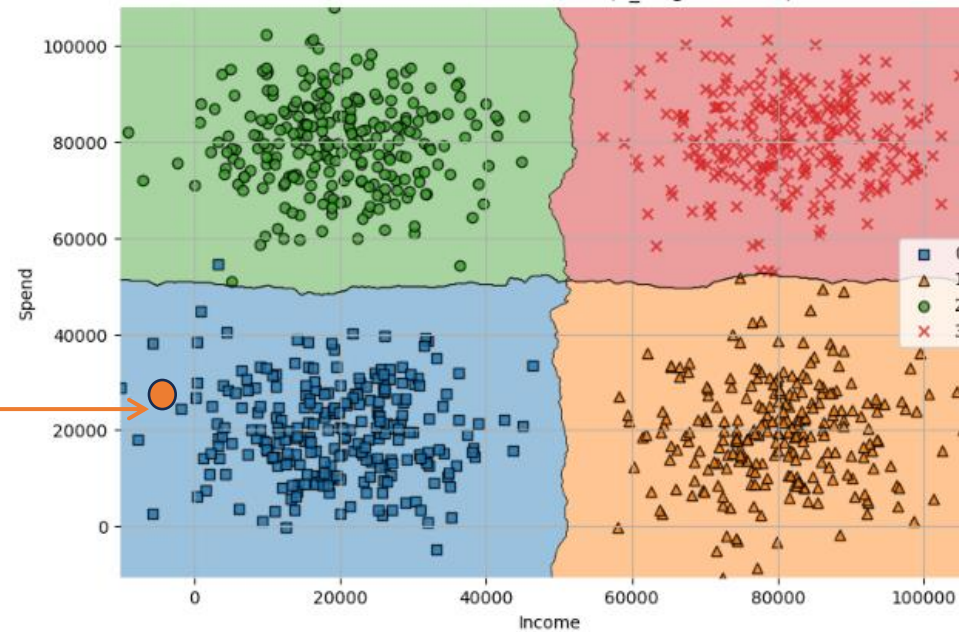
Output (Model)

Classified data using class
boundaries

How the decision boundaries help?

Predict
customer type
of a
New customer

Income	Spend
32805.917934	9034.764100



Output (Model)

Classified data using class
boundaries

Prediction
(customer type)
Class target

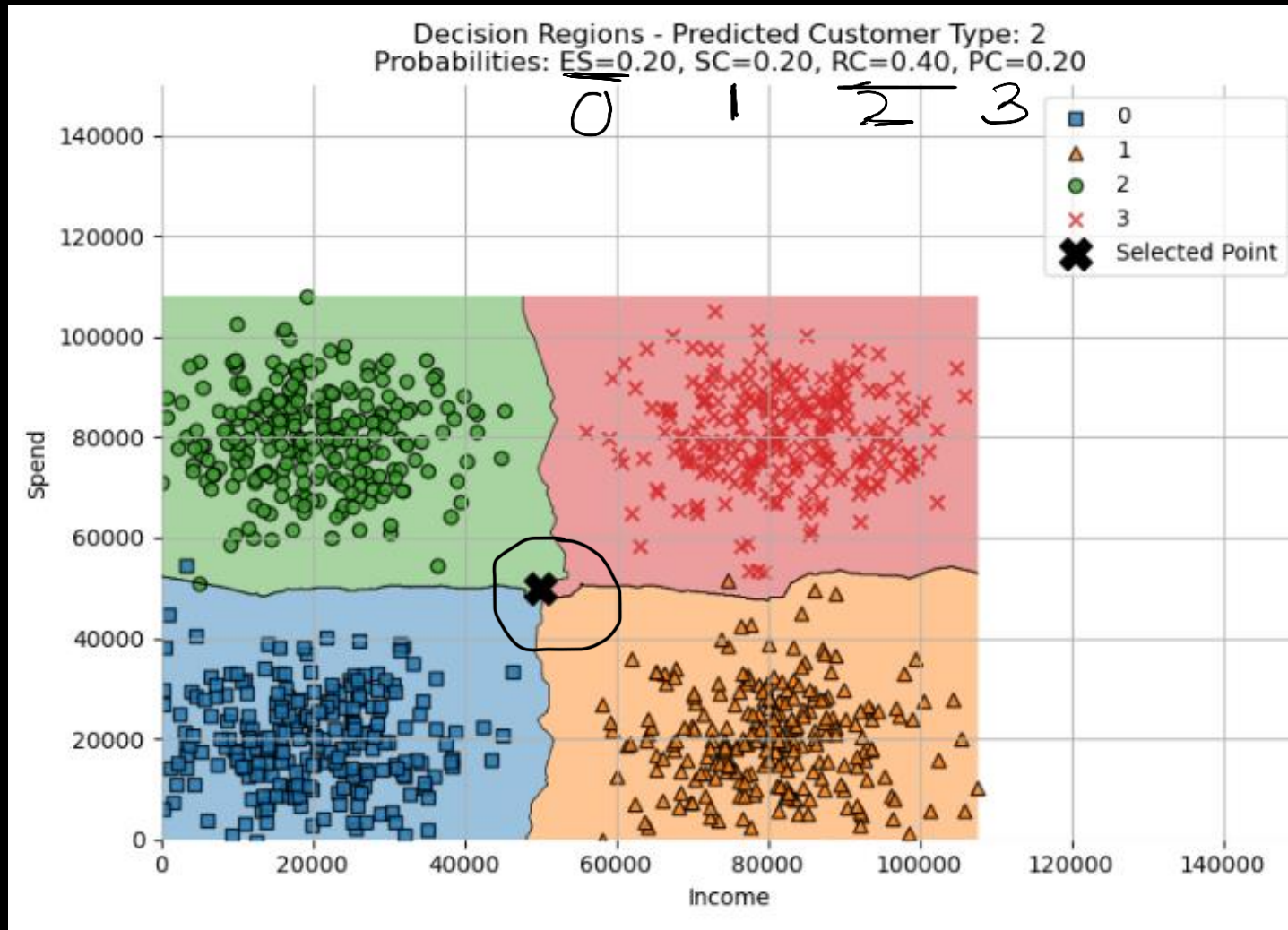
ES 0

Predicted Class: [0]

Prediction Probabilities:
[[0.6, 0.4, 0., 0.]]

i.e.
New customer is
closer/ more similar to
customers of ES type
than other types

Typical prediction



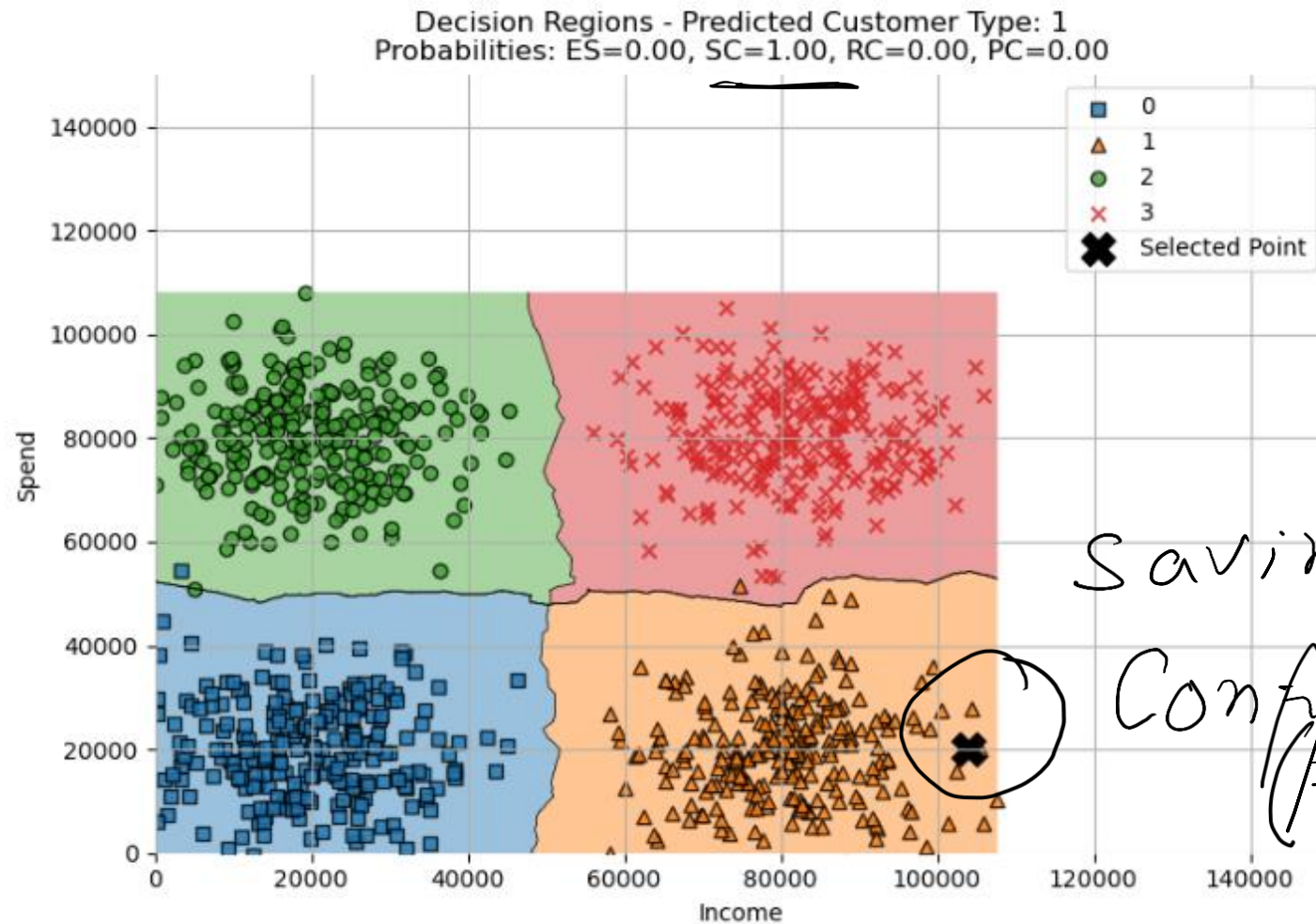
- Valid input data
 - Income : 0 to 100,000
 - Spend : 0 to 100,000
- New customer data

Income:

Spend:

Another customer

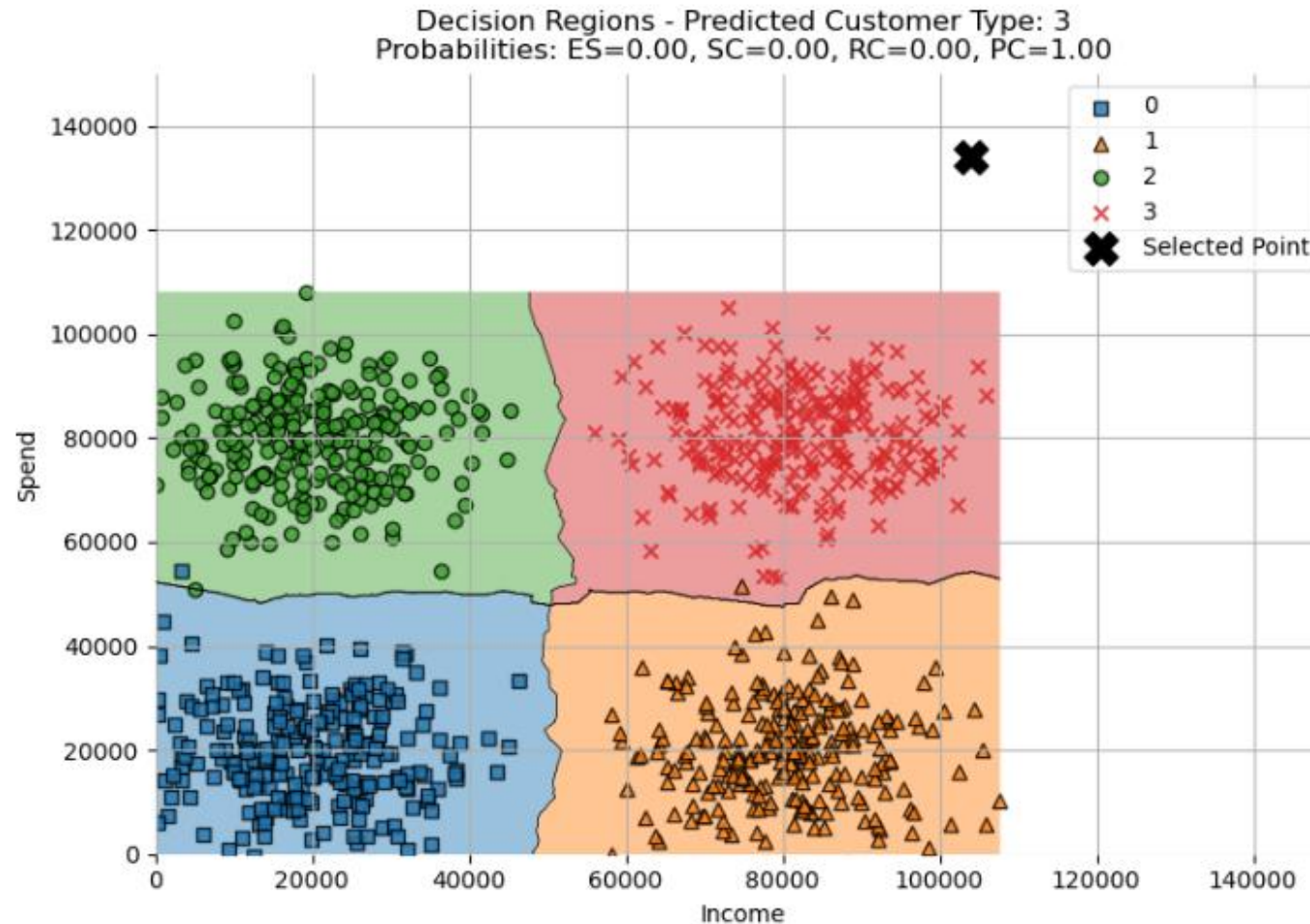
Income: 104000.00
Spend: 20000.00



Savings Type
Confident
(pred prob)

Odd customer

Income: 104000.00
Spend: 134000.00



Validation of Input
data for prediction is
required

(use of [inference
pipeline](#))

what if the prediction probabilities are same?

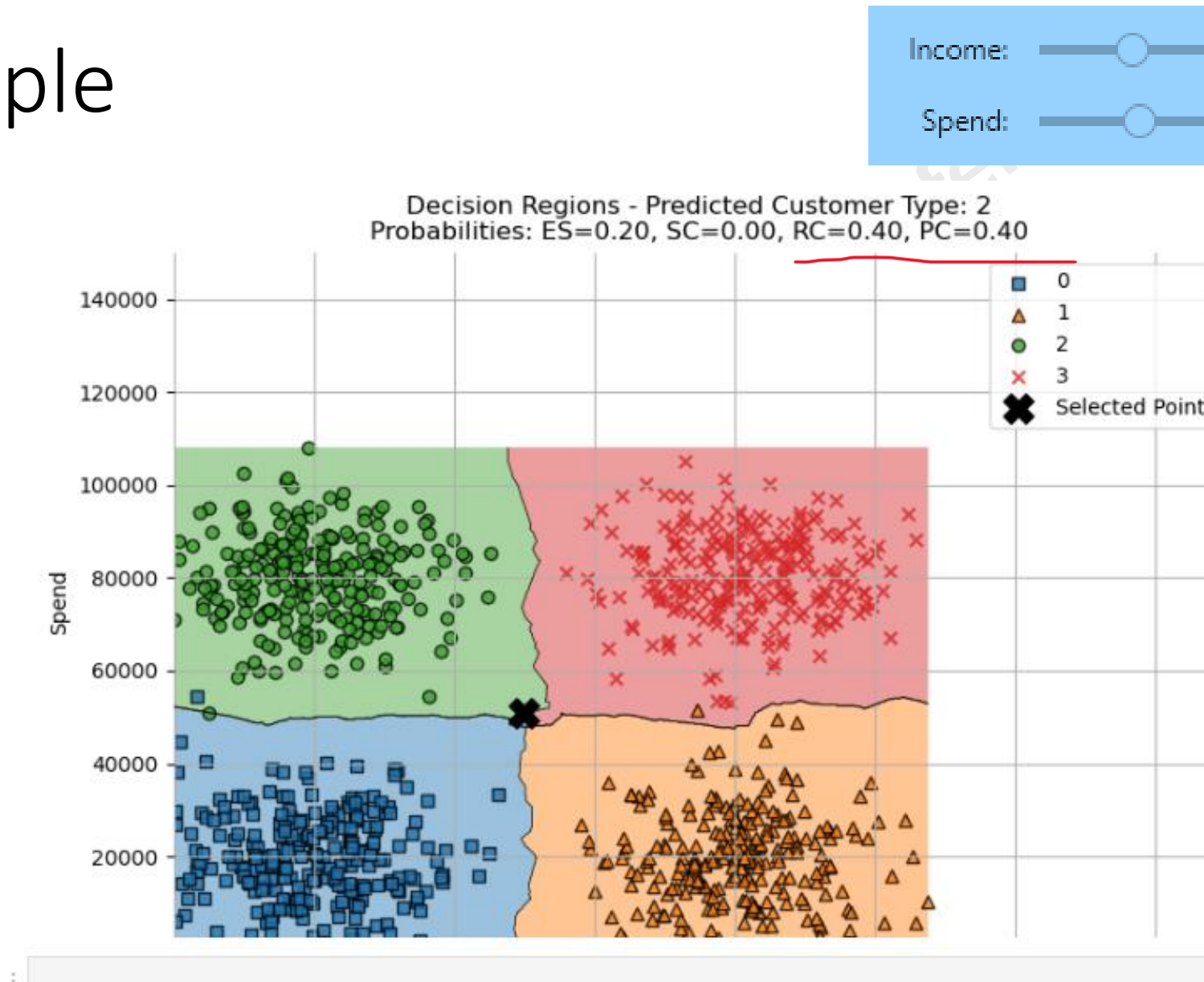
Model Uncertainty

- close probabilities suggest that the input might lie near the decision boundary between classes
- making it **challenging** for the model to confidently assign it to one specific class.

Feature Ambiguity

- input features may not provide clear distinctions between the classes
- Could mean noisy data, possibly

Example



ML algorithms use various different heuristics to break the tie

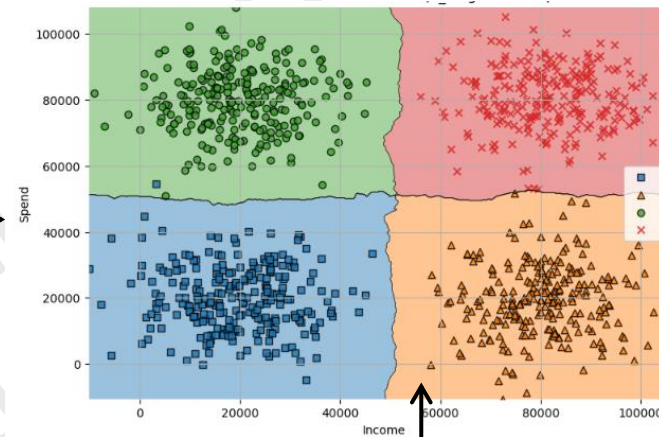
(some models do arbitrarily too)

But how good are the class boundaries? How to check?

Training Data :



ML Model:



Test Data :

	Income	Spend	Class	target
251	65284.796931	16704.322641	SC	1
536	7619.595991	85279.986831	RC	2
284	90165.273152	24076.408727	SC	1
111	8401.540579	28816.735024	ES	0
700	25389.476934	82954.129496	RC	2
502	14324.214015	85275.182847	RC	2

Actual label

Income	Spend
65284.796931	16704.322641
7619.595991	85279.986831
90165.273152	24076.408727
8401.540579	28816.735024
25389.476934	82954.129496
14324.214015	85275.182847

Compare

Class	target
ES	0
ES	0
ES	0
SC	1
RC	2
PC	3

Predicted label

Actual labels vs predicted labels

Test Data :
100 samples

	Income	Spend	Class	target
251	65284.796931	16704.322641	SC	1
536	7619.595991	85279.986831	RC	2
284	90165.273152	24076.408727	SC	1
111	8401.540579	28816.735024	ES	0
700	25389.476934	82954.129496	RC	2
502	14324.214015	85275.182847	RC	2

Actual label

Compare

Class	target
ES	0
ES	0
ES	0
SC	1
RC	2
PC	3

Predicted label

Number of
predictions =
100

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}}$$

$$\text{Accuracy} = \frac{90}{100} = 0.9$$

So, the accuracy of the model in this scenario is 0.9 or 90%.

Example

- Actual Labels (20 test samples) : `array([2, 3, 0, 2, 2, 3, 0, 0, 2, 1, 2, 2, 2, 2, 3, 0, 3, 3, 3, 2])`
- Predicted Labels : `array([1, 0, 1, 3, 3, 1, 1, 1, 3, 3, 0, 0, 3, 1, 1, 0, 3, 0, 0, 2])`

Accuracy: 15.00%

Other mismatches

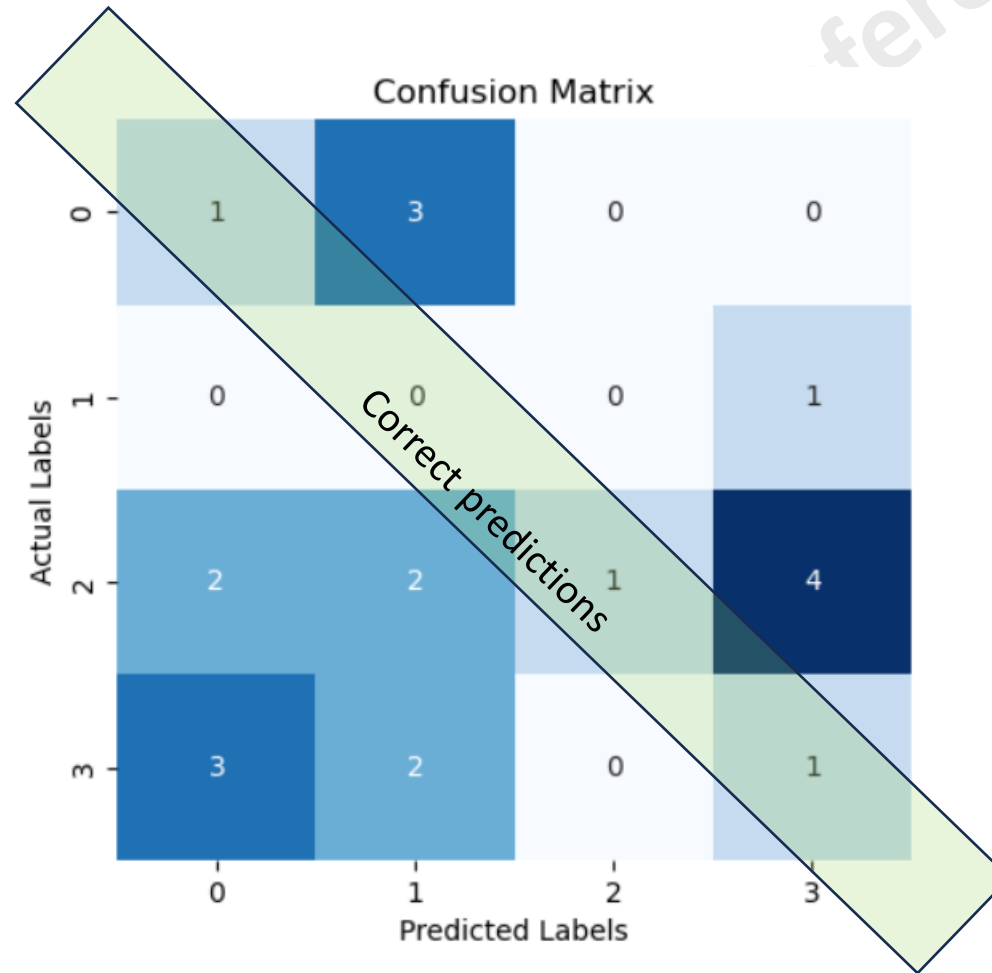
- Errors in prediction = $(1 - \text{accuracy}) = 85\%$

How is model treating the predictions (accuracy or error wise)?

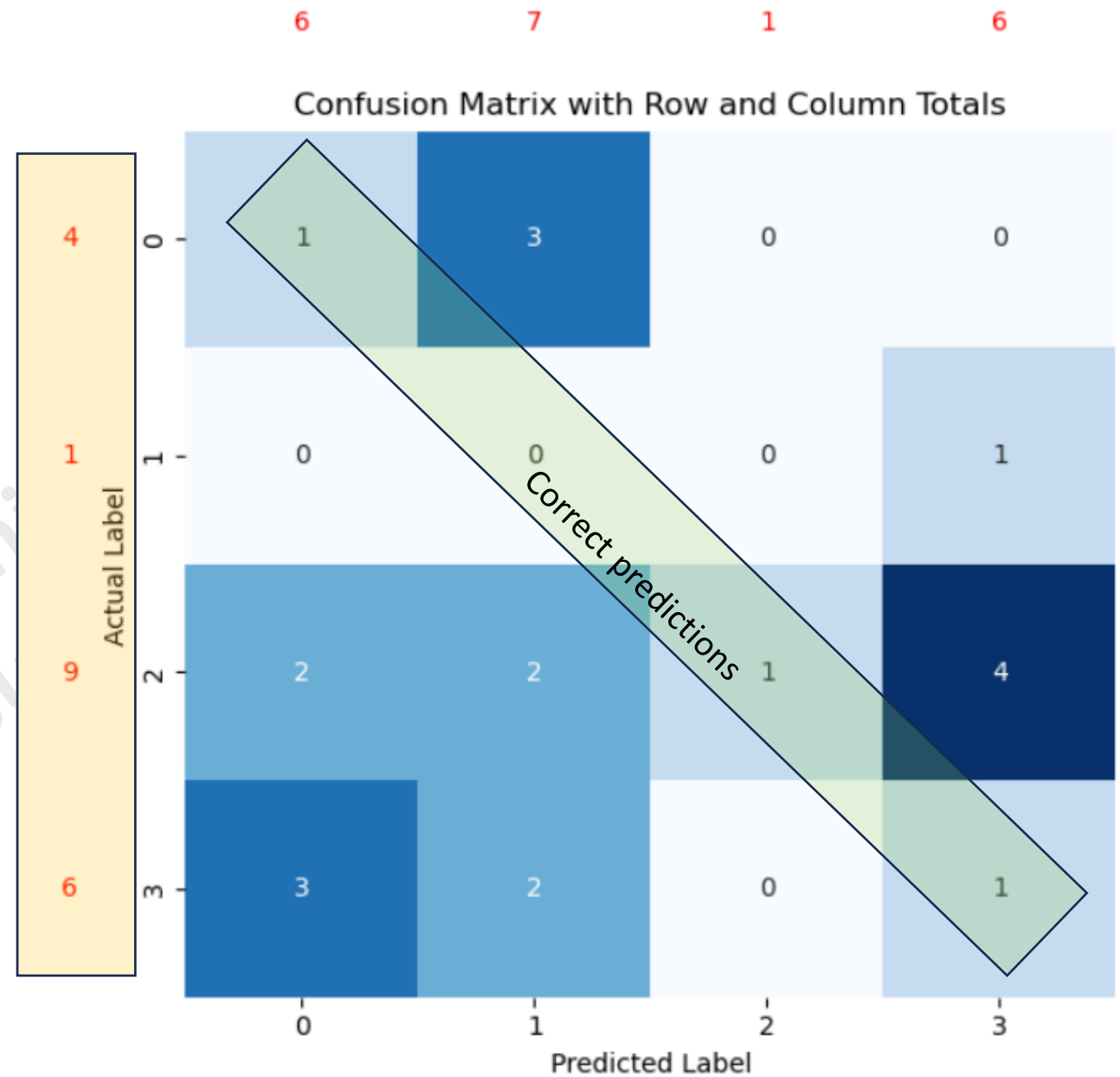
- This is measured as **confusion matrix**



Confusion matrix



Which classes are being predicted better?

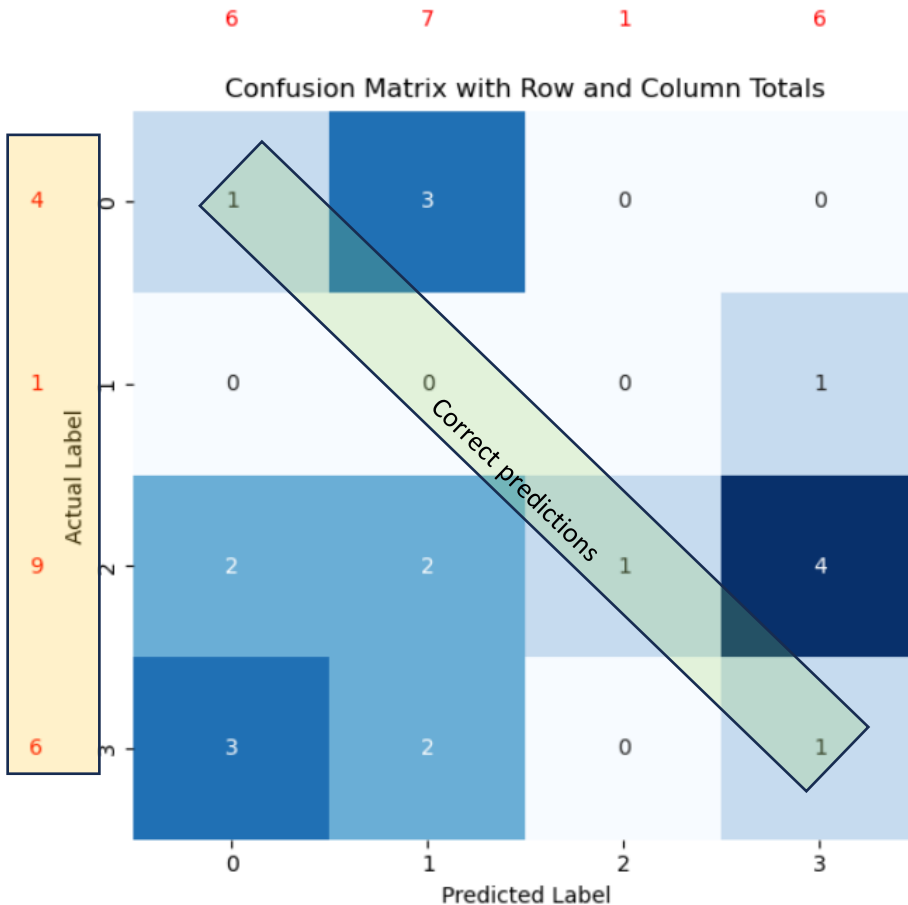


Recall



also known as **Sensitivity** or **True Positive Rate**.

$$\text{Recall}_{C_i} = \frac{\text{True Positives}_{C_i}}{\text{True Positives}_{C_i} + \text{False Negatives}_{C_i}}$$



- Class 0: Recall = 0.25
- Class 1: Recall = 0.00
- Class 2: Recall = 0.11
- Class 3: Recall = 0.17

True Positives_{C_i} is the number of instances correctly classified as class C_i .

False Negatives_{C_i} is the number of instances that actually belong to class C_i but were incorrectly classified as another class.

Do we test the model with training data also?

- Yes
- Let us see – why?

Training dataset (as test)

Income	Spend	Customer Type

Test dataset

Income	Spend	Customer Type

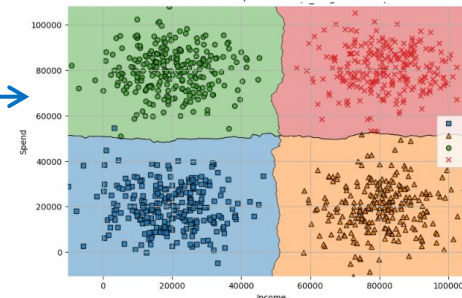
Evaluate
(train set)

Evaluate
(test set)

Training dataset

Income	Spend	Customer Type

Train the ML
model



Train (acc)	Test (acc)	meaning
95%	55%	Model overfitting, poor generalization
45%	43%	Underfitted model
84%	79%	Generalized model

How do we know we have enough training data?

Model Performance on Training Data

- model is performing well on the training data but poorly on unseen data (validation or test set), it might be overfitting.
- Overfitting occurs when the model learns the training data too well, capturing noise or random fluctuations rather than the underlying patterns.

Learning Curves

- Plotting learning curves, which show the model's performance on both the training and validation sets over time, can provide insights

Feature Importance Stability

- Assessing the stability of feature importance across different runs or subsets of the data can provide insights. If the importance of features changes drastically, it might suggest that the model is sensitive to variations in the training data.

Data Diversity

- Ensure that your dataset covers the diversity of scenarios that your model is likely to encounter in the real world. A lack of diversity can lead to biased models that do not generalize well.

Domain Knowledge

- Consider the complexity of the problem and the underlying data distribution. Complex tasks or nuanced patterns may require larger datasets.

Feature Engineering topics – to be covered later

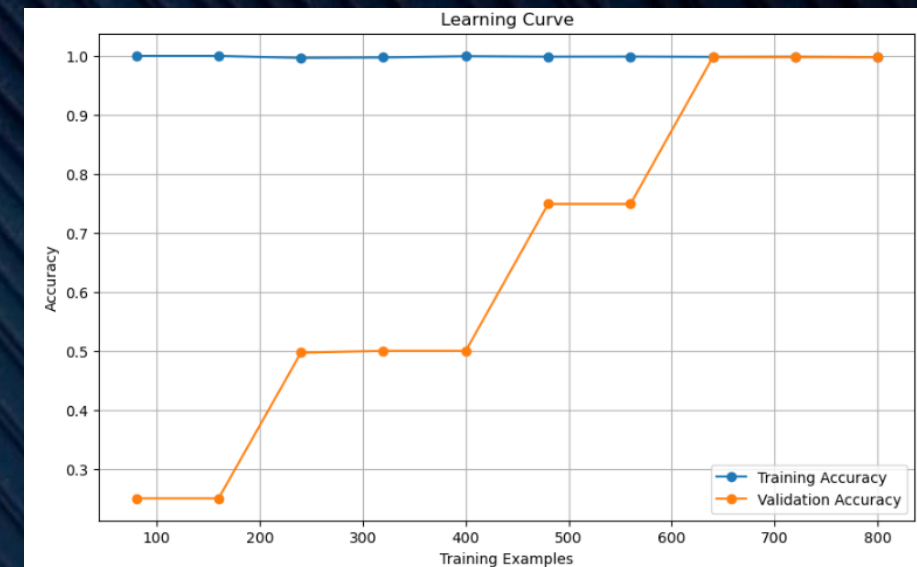
Example : model learning curve

Curves and axes

- **Training Accuracy Curve:** mean training accuracy across different training set sizes.
- **Validation Accuracy Curve:** mean validation accuracy across different training set sizes.
- **Training Set Size:** The x-axis represents the number of training examples
- **Accuracy Metrics: Accuracy:** The y-axis represents the accuracy

Interpretation:

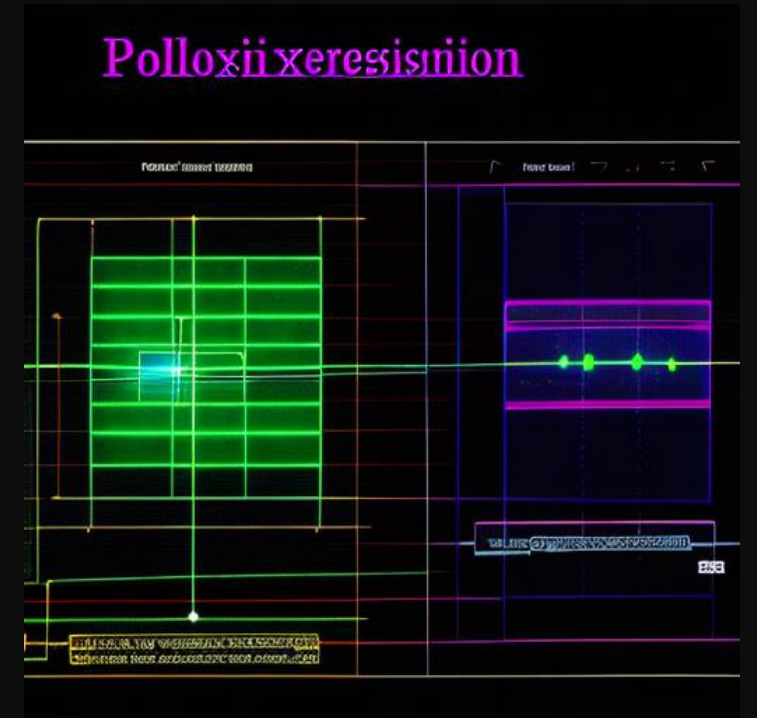
- **Underfitting (High Bias):** If both training and validation accuracies are low and close to each other, the model might be too simple and is **underfitting** the data.
- **Overfitting (High Variance):** If there's a significant gap between training and validation accuracies, the model may be too complex and is **overfitting** the training data.
- **Ideal Scenario:** model is performing well if the training and validation accuracies are both high, and there's a small gap between them.



Supervised models – regression

Simple linear regression
Multiple linear regression
Metrics
Assumptions

Easy
intuition



Why is it called regression?

01

"regression" - it was coined by Sir Francis Galton in the late 19th century.

02

Galton observed that the heights of children tended to "regress" towards the average height of their parents.

03

He introduced the term to describe the phenomenon where extreme values tend to move (or regress) towards the mean in subsequent generations.

Example: Height Regression

- Imagine a scenario where the heights (in inches) of parents are observed:
 - Father's height: 75 inches
 - Mother's height: 68 inches
- based on **Galton's** observation, if the average height in the population is, say, 70 inches, their children's heights might "regress" toward this average.
- Childrens' height
 - First child's height: 72 inches
 - Second child's height: 69 inches
 - Third child's height: 71 inches
- Observation
 - Even though the parents had extreme heights, the children's heights tend to move closer to the population mean.
 - phenomenon, termed "regression," was Galton's way of describing the tendency of extreme values to move towards the average over successive generations.

Example: House Price Prediction (ML type)

01

Consider a real estate scenario where we want to **predict house prices** based on features like square footage, number of bedrooms, and location.

02

use regression analysis to build a model that relates these features to the house price.

03

model, once trained, can predict the price of a new house given its features.

- Square footage: 2000 sq. ft.
- Number of bedrooms: 3
- Location: Suburb

Galton's Theory and Regression in Machine Learning

Galton's Observation:

- In the context of Galton's work on heredity and the heights of parents and children, he observed that extreme values (very tall or very short heights) tended to move or "regress" toward the average height of the population in subsequent generations.

Concept of Regression in Machine Learning:

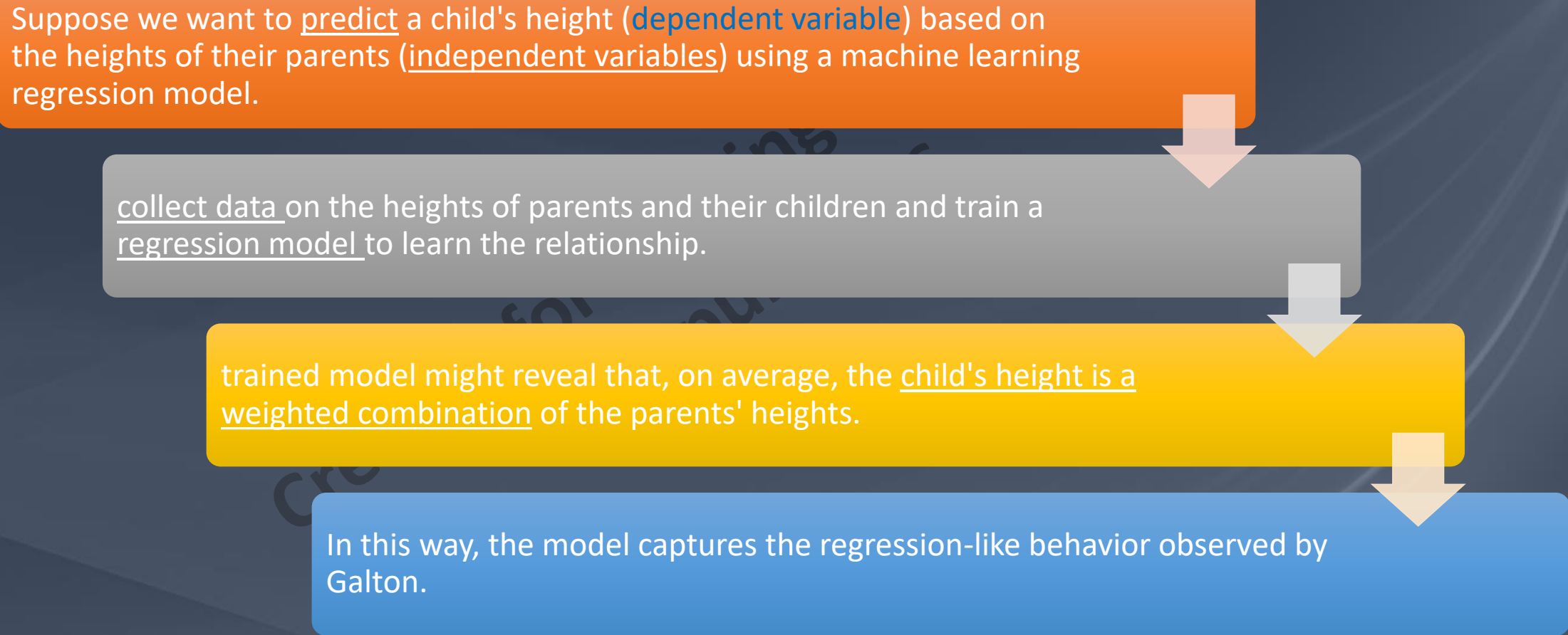
- In machine learning, regression refers to the statistical method used to model the relationship between variables.
- aims to understand how changes in one variable (**dependent variable**) are associated with changes in another or multiple variables (**independent variables**).

Connection:

- connection lies in the idea that both Galton's observation and regression in machine learning involve understanding the tendency of values to move toward some central point (average or mean).

Example: Predicting Heights (Machine Learning Perspective)

Suppose we want to predict a child's height (dependent variable) based on the heights of their parents (independent variables) using a machine learning regression model.



```
graph TD; A[Suppose we want to predict a child's height (dependent variable) based on the heights of their parents (independent variables) using a machine learning regression model.] --> B[collect data on the heights of parents and their children and train a regression model to learn the relationship.]; B --> C[trained model might reveal that, on average, the child's height is a weighted combination of the parents' heights.]; C --> D[In this way, the model captures the regression-like behavior observed by Galton.];
```

collect data on the heights of parents and their children and train a regression model to learn the relationship.

trained model might reveal that, on average, the child's height is a weighted combination of the parents' heights.

In this way, the model captures the regression-like behavior observed by Galton.

Understanding Relationships

Both Galton's regression concept and regression in machine learning share the common theme of understanding and modeling relationships between variables

Galton's idea of regression observed in genetics laid the conceptual groundwork for the broader statistical method of regression analysis used in machine learning

term "regression" in both cases captures the notion of values tending to move or "regress" toward some central tendency.

Problem statement

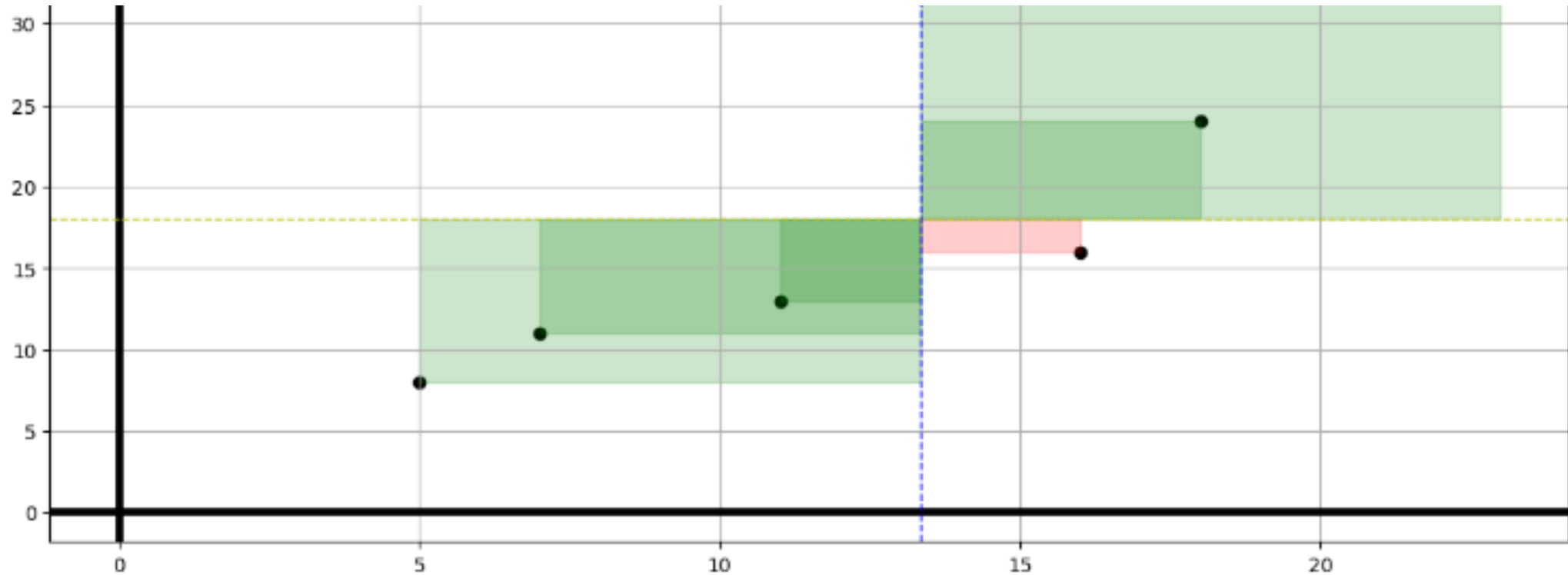
- Let us we have two columns
 - X -> Income (numeric)
 - Y -> Predicted savings (numeric)
- First, we need to establish Predicated Savings as weighted combination of Income

$$\text{Predicted Savings } (\hat{y}) = \text{Intercept} + \text{Slope} \cdot \text{Income}$$

slope = covariance / var(income)

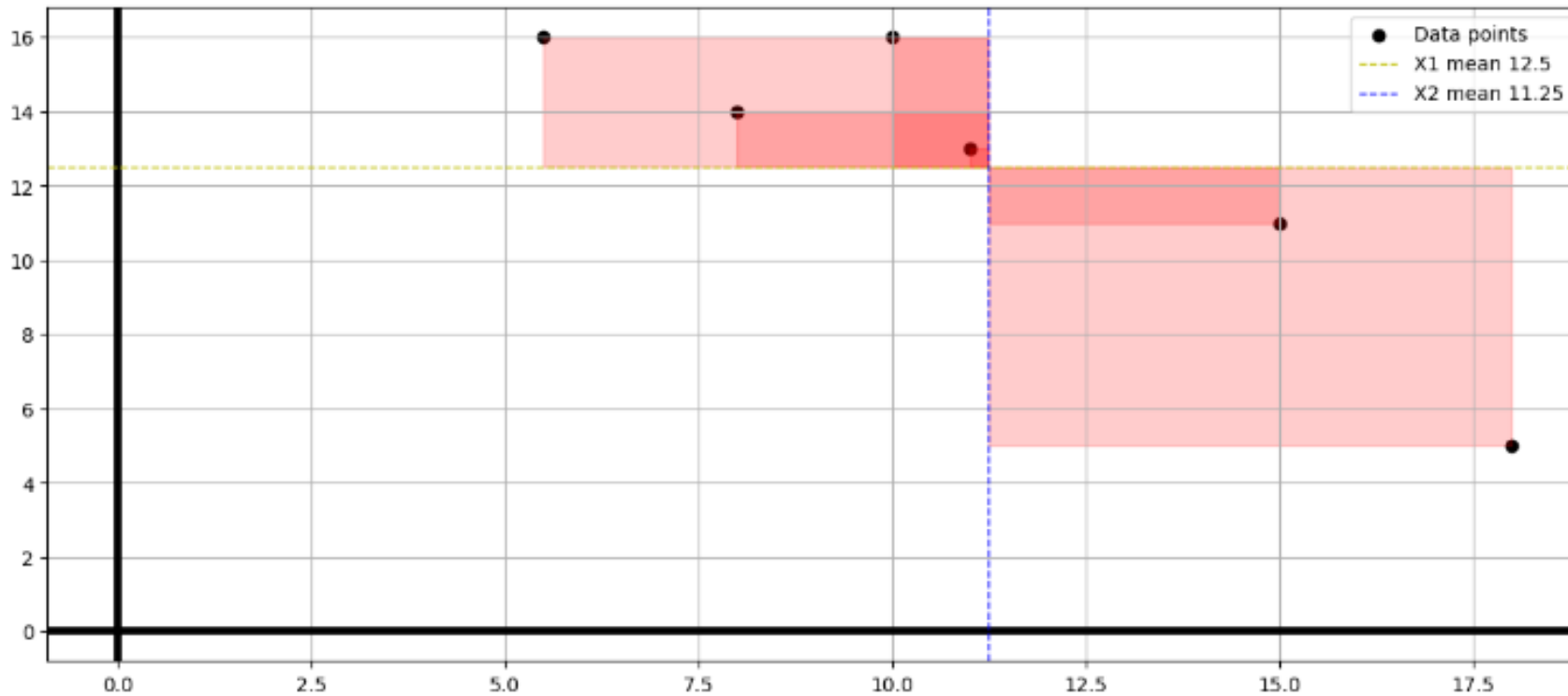
intercept = mean_savings - slope * mean_income

	x	y
0	5.488135	16.964428
1	7.151894	16.893471
2	6.027634	15.681403
3	5.448832	12.189472
4	4.236548	6.367116
5	6.458941	17.225119



Positive Covariance

- **Recall : COV**
- Covariance is a measure of how much two variables change together.
- If the covariance between two variables is positive, it indicates that when one variable is above (below) its mean, the other variable tends to be above (below) its mean as well.



Negative Covariance

- If the covariance between two variables is negative, it suggests that when one variable is above its mean, the other variable tends to be below its mean, and vice versa.

Slope

$$m = \frac{\text{Cov}(X,Y)}{\text{Var}(X)}$$

- $\text{Cov}(X,Y)$ is the covariance between the independent variable X and the dependent variable Y .
- $\text{Var}(X)$ is the variance of the independent variable X .

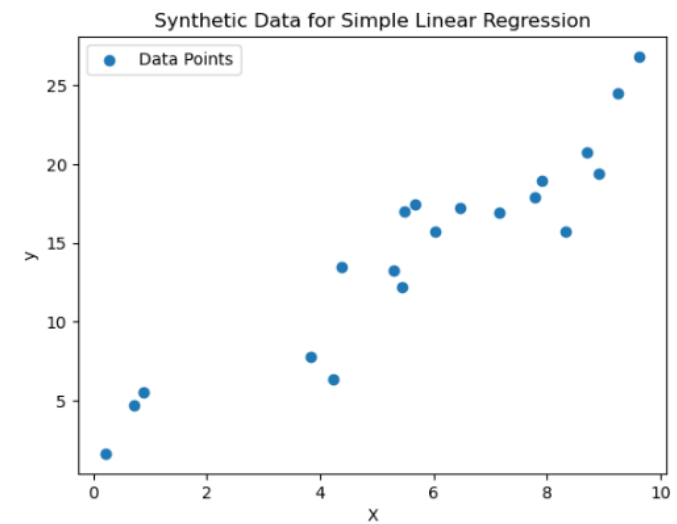
Slope (m):

- formula for the slope (m) essentially **normalizes** the covariance by the variance of X .
- normalization ensures that the scale of the variables does not impact the magnitude of the slope.
- If $\text{Cov}(X,Y)$ is large compared to $\text{Var}(X)$, the slope will be relatively steep, indicating a strong relationship between X and Y .
- If $\text{Cov}(X,Y)$ is small compared to $\text{Var}(X)$, the slope will be relatively shallow, indicating a weaker relationship.

Steps - data

- Linearly related samples

	x	y
0	5.488135	16.964428
1	7.151894	16.893471
2	6.027634	15.681403
3	5.448832	12.189472
4	4.236548	6.367116
5	6.458941	17.225119
6	4.375872	13.480617
7	8.917730	19.351130
8	9.636628	26.812764
9	3.834415	7.760099
10	7.917250	18.926018
11	5.288949	13.203531
12	5.680446	17.426450



Step 2 & 3

- Compute mean of X & y
 - Mean of X :
5.815548245225974
 - Mean of y :
14.643082378446588
- Compute deviations of each sample (X, y)

	X	y	deviations_X	deviations_y
	.488135	16.964428	-0.327413	2.321346
	7.151894	16.893471	1.336345	2.250388
	6.027634	15.681403	0.212086	1.038321
3	5.448832	12.189472	-0.366716	-2.453610
4	4.236548	6.367116	-1.579000	-8.275966
5	6.458941	17.225119	0.643393	2.582037
6	4.375872	13.480617	-1.439676	-1.162466
7	8.917730	19.351130	3.102182	4.708048
8	9.636628	26.812764	3.821079	12.169682
	3.834415	7.760099	-1.981133	-6.882983
	1.017250	10.026010	2.101702	4.382025

Compute covariance

`df['covariance'] = df['deviations_X'] *
df['deviations_y']`

	X	y	deviations_X	deviations_y	covariance
0	5.488135	16.964428	-0.327413	2.321346	-0.760039
1	7.151894	16.893471	1.336345	2.250388	3.007296
2	6.027634	15.681403	0.212086	1.038321	0.220213
3	5.448832	12.189472	-0.366716	-2.453610	0.899779
4	4.236548	6.367116	-1.579000	-8.275966	13.067752
5	6.458941	17.225119	0.643393	2.582037	1.661264
6	4.375872	13.480617	-1.439676	-1.162466	1.673574
7	8.917730	19.351130	3.102182	4.708048	14.605219
8	9.636628	26.812764	3.821079	12.169682	46.501321
9	3.834415	7.760099	-1.981133	-6.882983	13.636106
10	7.917250	18.926018	2.101702	4.282935	9.001455
11	5.288949	13.203531	-0.526599	-1.439552	0.758067
12	5.680446	17.426450	-0.135103	2.783367	-0.376040

slope

$$\hat{y} = \text{Intercept} + \text{Slope} \cdot x$$

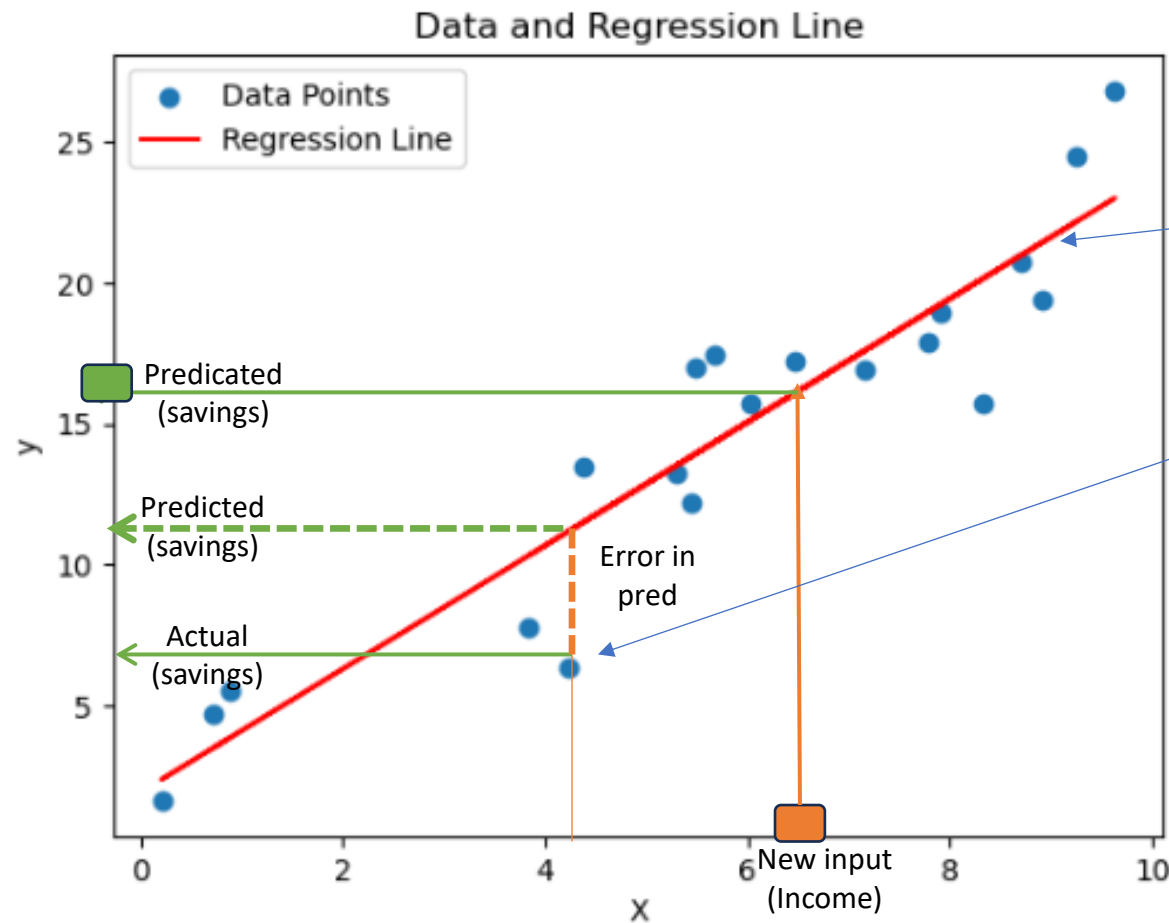
```
slope      = covariance / np.var(income)
intercept  = mean_savings - slope * mean_income
```

```
# Compute slope (coefficient) and intercept for the linear regression model
slope = covariance / np.var(df['X'])
intercept = mean_y - slope * mean_X
```

```
print("Intercept:", intercept)
print("Slope:", slope)
```

```
Intercept: 1.92458157807868
Slope: 2.1869822524142273
```

Model



`regression_line` -> represents the linear relationship between the independent variable (X) and the dependent variable (y)

What if the income is exactly this?

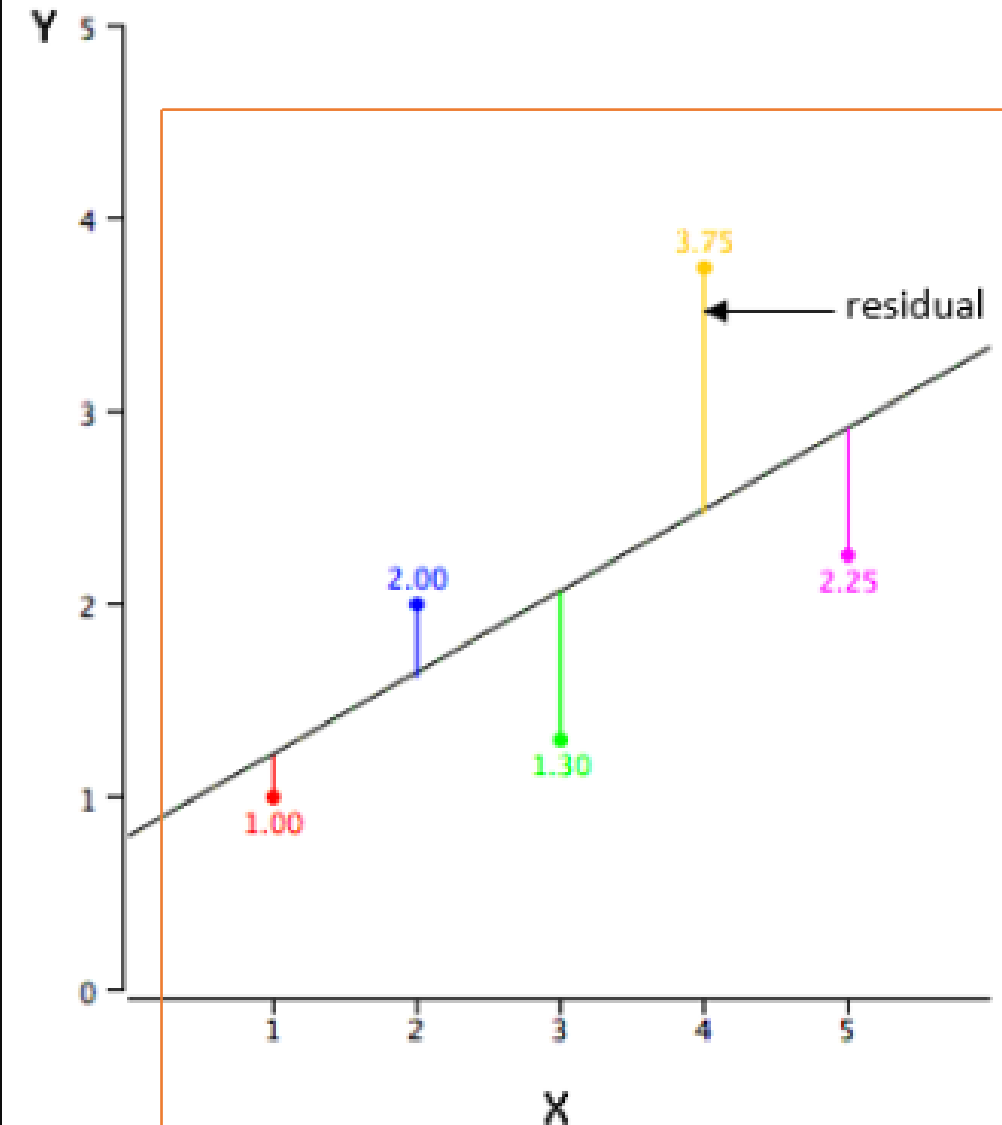


need to keep the pred
errors to minimum

- **Objective of Regression Analysis:**
- primary goal of regression analysis is to build a model that accurately captures the underlying relationship between variables.
- this involves finding the slope and intercept that minimize the sum of prediction errors.
- Errors define the goodness of the regression model
- Regression metrics are based on ERRORS

Regression metrics

- Mean-Absolute-Error(MAE).
- Mean Squared Error(MSE)
- Root-Mean-Squared-Error(RMSE).
- Mean Absolute Percentage Error (MAPE)
- Mean Percentage Error (MPE)
- R^2 or Coefficient of Determination.
- Adjusted R^2



$$MAE = \frac{1}{n} \sum |y - \hat{y}|$$

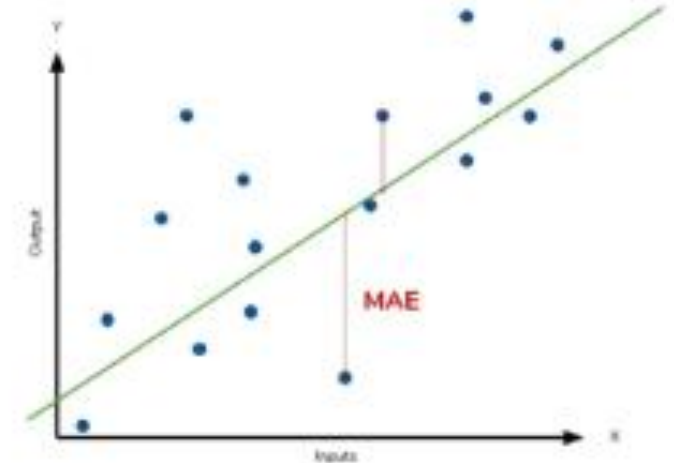
Divide by the total number of data points

Predicted output value

Actual output value

Sum of

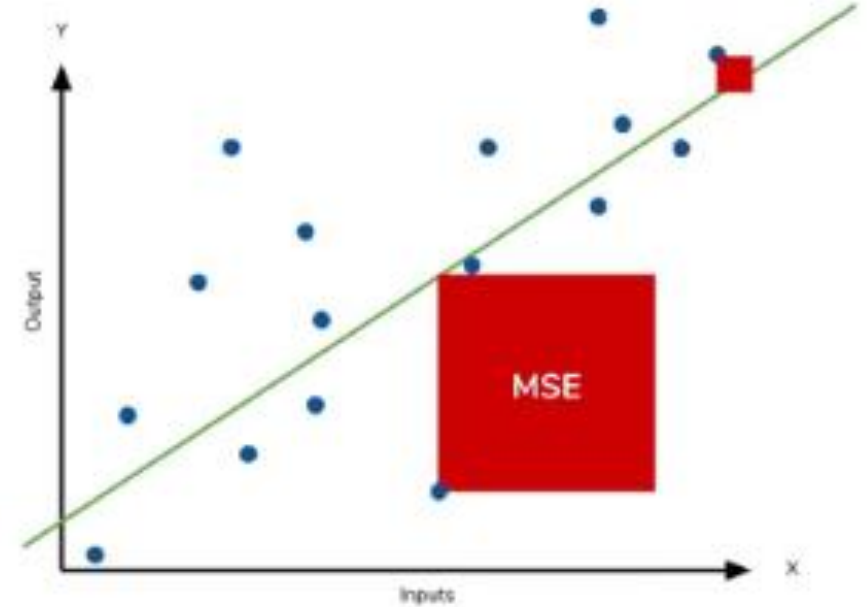
The absolute value of the residual



Mean-Absolute-Error(MAE)

- Calculate the residual for every data point, taking only the absolute value of each
- Take the average of all these residuals.
- MAE describes the typical magnitude of the residuals.
- small MAE suggests the model is great at prediction, while a large MAE suggests that your model may have trouble in certain areas.
- A MAE of 0 means that your model is a perfect predictor of the outputs (but this will almost never happen).

$$MSE = \frac{1}{n} \sum \underbrace{\left(y - \hat{y} \right)^2}_{\text{The square of the difference between actual and predicted}}$$



Mean square error (MSE)

- The mean square error (MSE) is just like the MAE but squares the difference before summing them all instead of using the absolute value.
- Because we are squaring the difference, the MSE will almost always be bigger than the MAE.
- effect of the square term in the MSE equation is most apparent with the **presence of outliers** in our data.

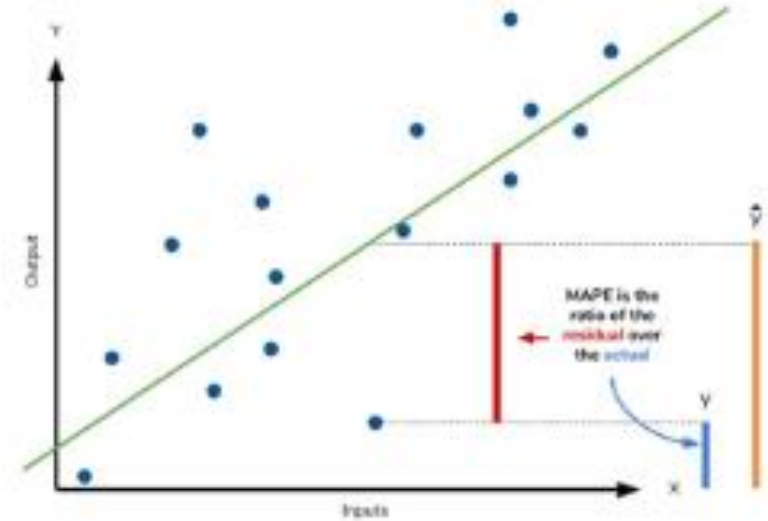
Root mean squared error (RMSE)

- Square root of the MSE
- Since the MSE and RMSE both square the residual, they are similarly affected by outliers.
- The RMSE is analogous to the standard deviation (MSE to variance)
- Is a measure of how large your residuals are spread out.



$$MAPE = \frac{100\%}{n} \sum \left| \frac{\overbrace{y - \hat{y}}^{\text{The residual}}}{\underbrace{y}_{\text{Each residual is scaled against the actual value}}} \right|$$

Multiplying by 100% converts to percentage



Mean absolute percentage error (MAPE)

- mean absolute percentage error (MAPE) is the percentage equivalent of MAE.
- equation looks just like that of MAE, but with adjustments to convert everything into percentages.
- Both MAPE and MAE are robust to the effects of outliers

$$MAPE = \frac{100\%}{n} \sum \left| \frac{y - \hat{y}}{y} \right|$$

Multiplying by 100% converts to percentage

The residual

Each residual is scaled against the actual value

$\hat{y}$ is smaller than the actual value

$n = 1 \quad \hat{y} = 10 \quad y = 20$

MAPE = 50%

$\hat{y}$ is greater than the actual value

$n = 1 \quad \hat{y} = 20 \quad y = 10$

MAPE = 100%

Key points on MAPE

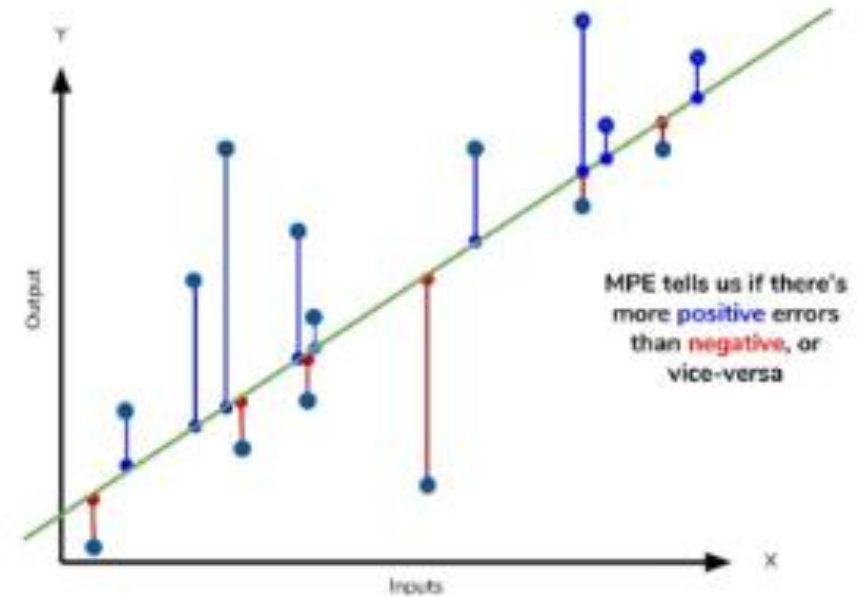
- MAPE's weaknesses actually stem from **use of division operation**.
- MAPE can **grow unexpectedly** large if the actual values are exceptionally small themselves.
- the MAPE is biased towards predictions that are systematically less than the actual values themselves. That is to say, MAPE will be lower when the prediction is lower than the actual compared to a prediction that is higher by the same amount.

$$MPE = \frac{100\%}{n} \sum \left(\frac{y - \hat{y}}{y} \right)$$

$$MAPE = \frac{100\%}{n} \sum \left| \frac{y - \hat{y}}{y} \right|$$

Annotations for MAPE:

- Multiplying by 100% converts to percent output
- The residual $y - \hat{y}$
- Each residual is scaled against the actual value



Mean percentage error (MPE)

- mean percentage error (MPE) equation is exactly like that of MAPE.
- only difference is that it lacks the absolute value operation.
- Since positive and negative errors will cancel out, we cannot make any statements about how well the model predictions perform overall.
- if there are more negative or positive errors, this bias will show up in the MPE.
- Unlike MAE and MAPE, MPE is useful to us because it allows us to see if our model systematically underestimates (more negative error) or overestimates (positive error).

R^2

01

how close the data are to the fitted regression line.

02

signifies percentage of variations in the response variable that can be explained by the model.

03

$R\text{-squared} = \frac{\text{Explained variation}}{\text{Total variation}}$

04

Total variation is variation of response variable around its mean.

05

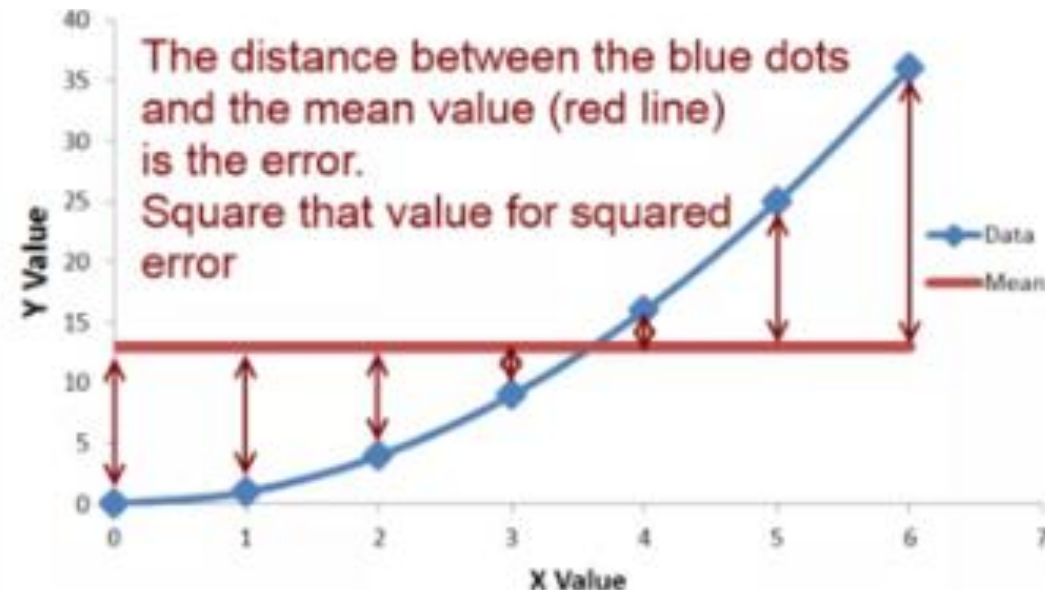
The closer the r -square to 100%, the better is the model.

Mean Value Of Data



$$MSE(\text{baseline}) = SST = \frac{1}{N} \sum_{i=1}^n (y - \bar{y})^2$$

Sum Squared Total Error



intuitive understanding

- can be thought of as the MSE that the simplest possible model would get.
- The simplest possible model would be to always predict the average of all samples
- First calculate how much error you would have if you don't even try to do regression, and instead just guess the mean of all the values.
- That is the total error. This results in summed squared error (SST)

Regression model

- The ratio of the **regression error** against the **total error** tells how much of the total error remains in your regression model.
- Subtracting that ratio from 1.0 gives how much error you removed using the regression analysis.
- That is R^2 !!!
- is the ratio between how good our model is vs how good is the naive mean model.
- R^2 gives an idea of how many data points fall within the line of the regression equation

$$R^2 = 1 - \frac{SSE}{SST}$$

$$R^2 = 1 - (ExplainedVariation/TotalVariation)$$

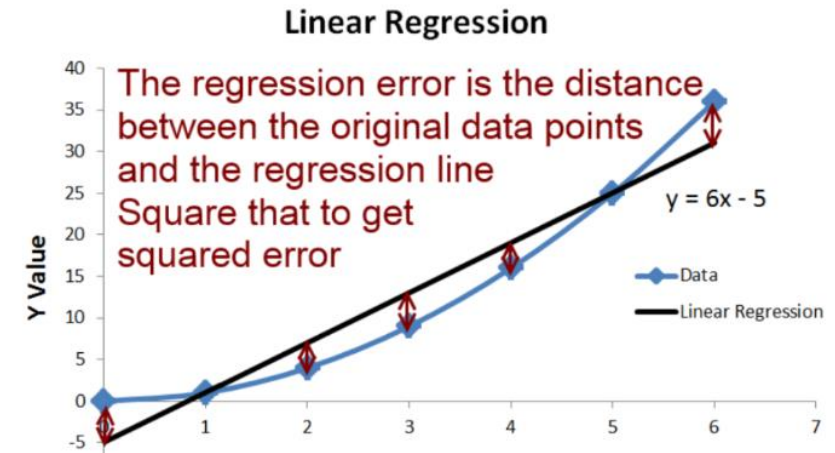
e.g.

say $SST = 50$ and $SSE = 40$

$$\text{so } \frac{SSE}{SST} = 40/50 = 0.8$$

this means 80% of the variation of y exists in the residuals. (in the model) i.e not explained by the variation of the predictor(s)

$$\text{So } R^2 = 1 - 0.8 = 0.2$$



How do we handle multiple predictors?

- Not in the same way that we saw in the earlier slides (SLR)
- SCIKIT learn uses the closed-form solution

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n + \varepsilon$$

Here:

- Y is the dependent variable.
- β_0 is the intercept.
- $\beta_1, \beta_2, \dots, \beta_n$ are the coefficients for the independent variables $X_1, X_2, \dots, X_n$ respectively.

$$\beta = (X^T X)^{-1} X^T y$$

To be covered in detail when we talk about mathematical formulation of a typical regression problem



Assumptions of linear regression

Linearity

Normality

Homoscedascity

Multi-collinearity



Linearity

The linearity assumption in linear regression implies that the relationship between the independent variable(s) and the dependent variable can be adequately represented by a straight line.

In other words, a change in the independent variable(s) should result in a proportional change in the dependent variable.

Example

Let's consider a **simple linear regression** with one independent variable (X) and one dependent variable (Y).

relationship is assumed to be of the form: $Y = \beta_0 + \beta_1 \cdot X$, where β_0 is the intercept, β_1 is the slope

Example: Salary Prediction

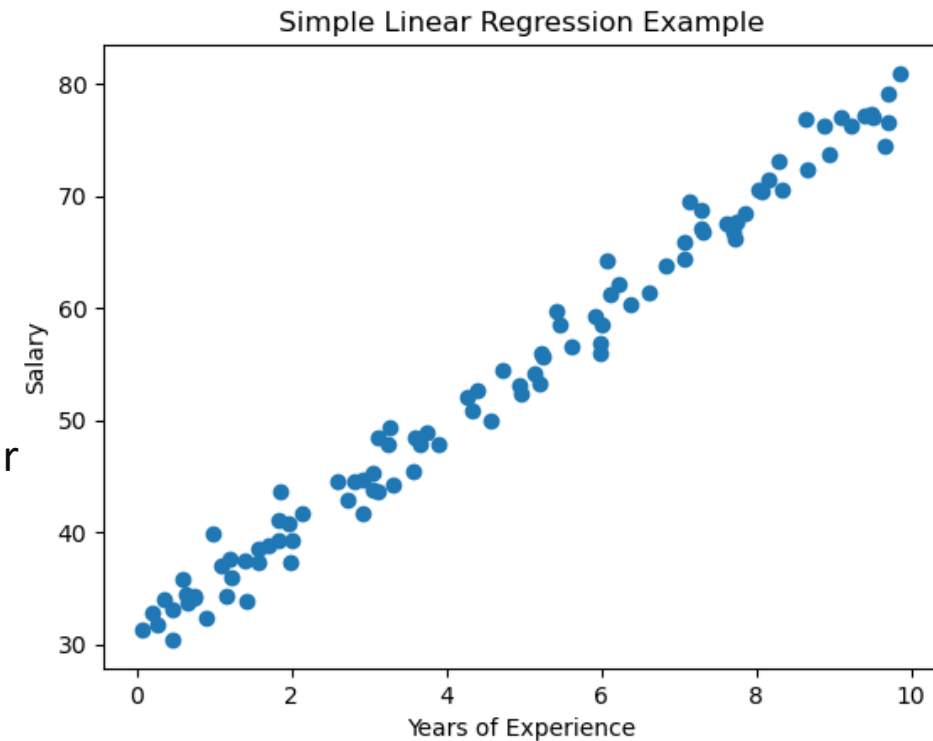
Independent Variable (X): Years of Experience

Dependent Variable (Y): Salary

If the linearity assumption holds, it implies that for each additional year of experience (ΔX), the change in salary (ΔY) is proportional.

salary increases by \$5,000 for every additional year of experience, the relationship is linear.

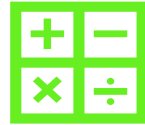
If the relationship is not linear, it might mean that salary increases at a decreasing rate or has some other non-linear pattern.



Example



linearity assumption extends to the combined effect of these variables on the dependent variable.



Example: Housing Price Prediction

Independent Variables (X1, X2, X3): Square Footage, Number of Bedrooms, Distance to City Center

Dependent Variable (Y): House Price



linearity assumption suggests that a linear combination of the effects of square footage, number of bedrooms, and distance to the city center should explain the variation in house prices.



For instance, if a one-unit increase in square footage is associated with a certain increase in house price, and the same holds for bedrooms and distance, the overall relationship is linear.

Homoscedascity

crucial for the reliability of statistical inferences and predictions made by linear regression models.

implies that the variability of errors is constant for all values of the independent variable(s), indicating that the spread of residuals does not change as the predictor(s) change.

examples

- **Scatterplot with Residuals:**

- *Homoscedasticity*: In a scatterplot of residuals against predicted values, the spread of points should form a roughly constant band, indicating homoscedasticity.
- *Heteroscedasticity*: A cone-shaped or funnel-shaped pattern in the scatterplot suggests varying spread, indicating heteroscedasticity.





Homoscedasticity: In a linear regression model predicting stock returns based on financial ratios, homoscedasticity would imply that the variability in prediction errors remains constant across different levels of financial ratios.



Heteroscedasticity: If the spread of residuals widens or narrows as financial ratios increase, it violates the assumption of homoscedasticity.

Financial Data Example



Transformations: Applying data transformations (e.g., log transformations) to stabilize the variance.



More focus on feature engineering :
column selection etc



Outlier Identification and Removal:
Identifying and addressing influential outliers that may contribute to heteroscedasticity.

Methods to Address Violations

Multi-collinearity

1

Multi-collinearity is a statistical phenomenon in linear regression that arises when two or more independent variables in a model are highly correlated.

2

This condition poses challenges in terms of interpreting the individual coefficients and standard errors of the regression model

Indicators of Multi-collinearity

High Correlation Coefficients: A correlation matrix showing high correlations (close to +1 or -1) between pairs of independent variables indicates potential multi-collinearity.

Variance Inflation Factor (VIF): VIF measures how much the variance of an estimated regression coefficient increases if your predictors are correlated. A high VIF (typically above 10) suggests multi-collinearity.

Consequences of Multi-collinearity

01

Unreliable Coefficients:
Multi-collinearity makes it challenging to determine the individual impact of each predictor on the dependent variable.

02

Increased Standard Errors: Coefficient estimates become less precise, leading to wider confidence intervals and potentially inaccurate hypothesis tests.

03

Inverted Relationships:
The signs of coefficients may be opposite to expectations due to the difficulty in disentangling the effects of correlated variables.

Methods to Address Multi-collinearity

Variable Selection: Choose a subset of variables that are most relevant to the research question, based on domain knowledge or statistical techniques.

Principal Component Analysis (PCA): Transform the original variables into a new set of uncorrelated variables

Ridge Regression or LASSO Regression: Regularization techniques

Collect More Data: Increasing the sample size may help mitigate the impact of multi-collinearity.

Unsupervised models

Concepts

Metrics

Easy
intuition

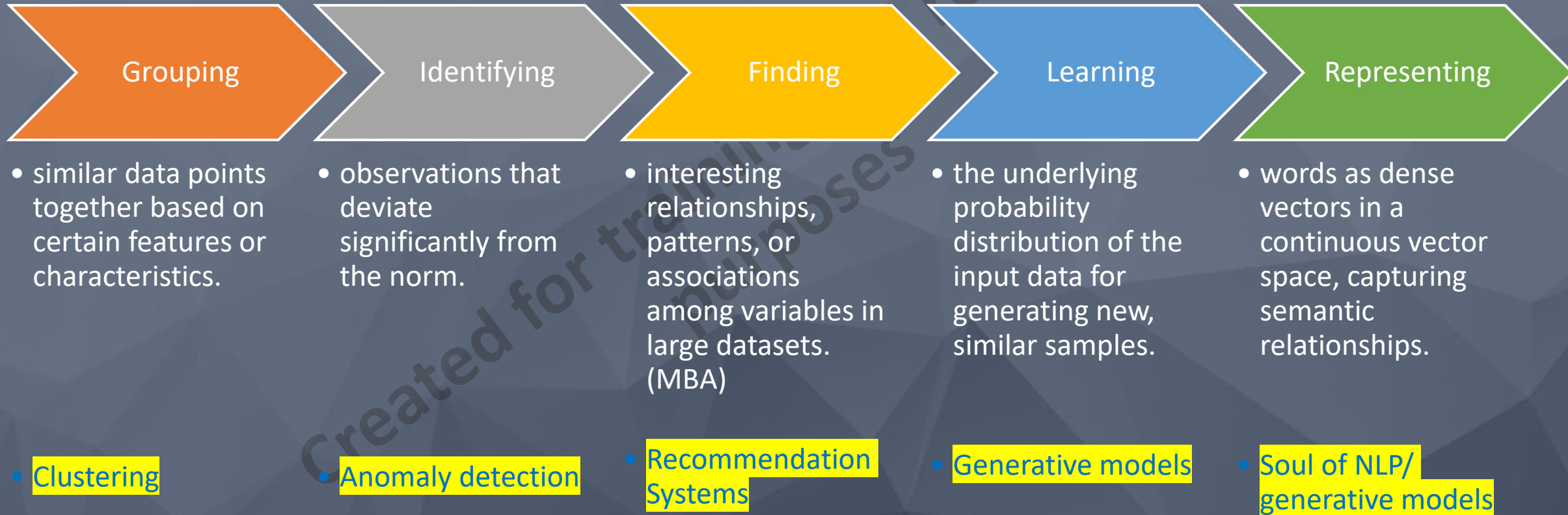


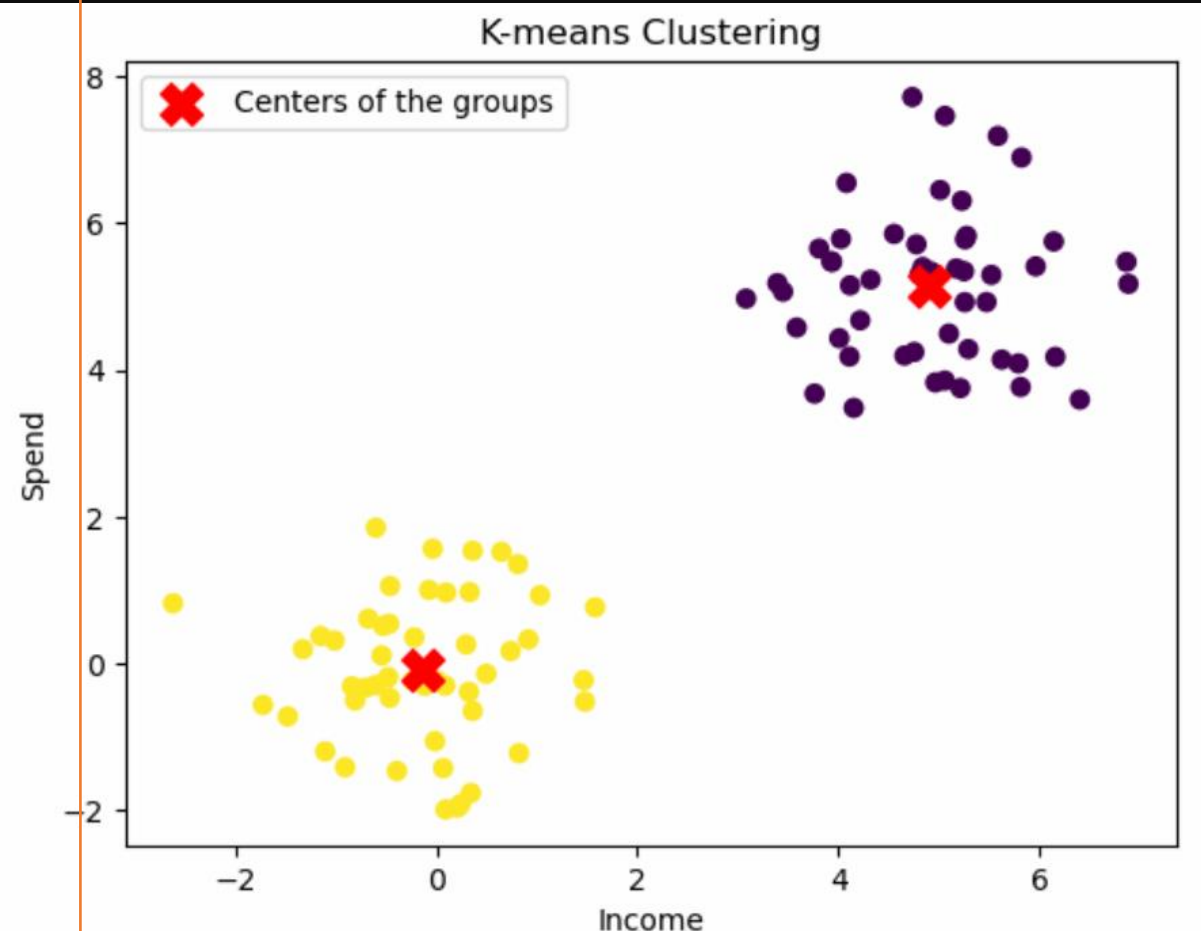
Concepts

Unsupervised learning is a category of machine learning where the purpose is to learn presence of units , segments or groups

The system tries to learn the patterns and structure from the data without any labeled responses

Purpose





Grouping - examples

- grouping similar data points.
- Similarity is based on various distance measures



Clustering Models

01

Think of it like sorting different types of fruits into separate baskets without knowing what each fruit is.

02

The algorithm groups similar things together without any guidance on what those groups should be.

03

these models help to organize and simplify data without knowing what the "right" organization should be. They're like finding patterns or creating summaries without a teacher telling them what to look for.

Evaluating an unsupervised model

01

Unsupervised learning involves tasks where the model is not provided with labeled training data, meaning there are no explicit target outputs for the input data.

02

the algorithm is not guided by a predefined set of correct answers during the training process.

03

lack of labeled data makes it challenging to define a clear objective function or ground truth for evaluation.

Labeled training data



Classification:

Task: Classifying emails as spam or non-spam.

Labeled Data: Each email in the training dataset is labeled as either "spam" or "non-spam."



Regression:

Task: Predicting the price of a house based on its features.

Labeled Data: Each example includes the features of a house (e.g., number of bedrooms, square footage) along with the corresponding sale price.



Image Classification:

Task: Identifying objects in images.

Labeled Data: For each image, there is a corresponding label indicating the object present (e.g., "cat," "dog," "car").



Natural Language Processing (NLP) - Sentiment Analysis:

Task: Determining the sentiment (positive, negative, neutral) of a movie review.

Labeled Data: Each movie review is labeled with its corresponding sentiment category.

Labeled training data



Object Detection:

Task: Detecting and localizing objects within an image.

Labeled Data: Each image is annotated with bounding boxes around the objects, along with the class labels for each object.



Medical Diagnosis:

Task: Predicting whether a patient has a certain medical condition based on diagnostic test results.

Labeled Data: Patient records include the results of diagnostic tests along with the binary label indicating the presence or absence of the medical condition.



Handwriting Recognition:

Task: Recognizing handwritten characters.

Labeled Data: Each image of a handwritten character is associated with the correct character label.



Fraud Detection:

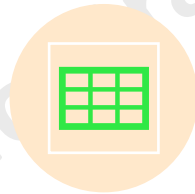
Task: Identifying fraudulent transactions.

Labeled Data: Each financial transaction is labeled as either "fraudulent" or "non-fraudulent."

Ways to evaluate



Inertia or Sum of Squared Distances (SSD): For clustering algorithms, such as k-means, you can use the inertia or SSD, which measures the sum of squared distances between each data point and its assigned cluster's centroid. Lower inertia values generally indicate better-defined clusters.

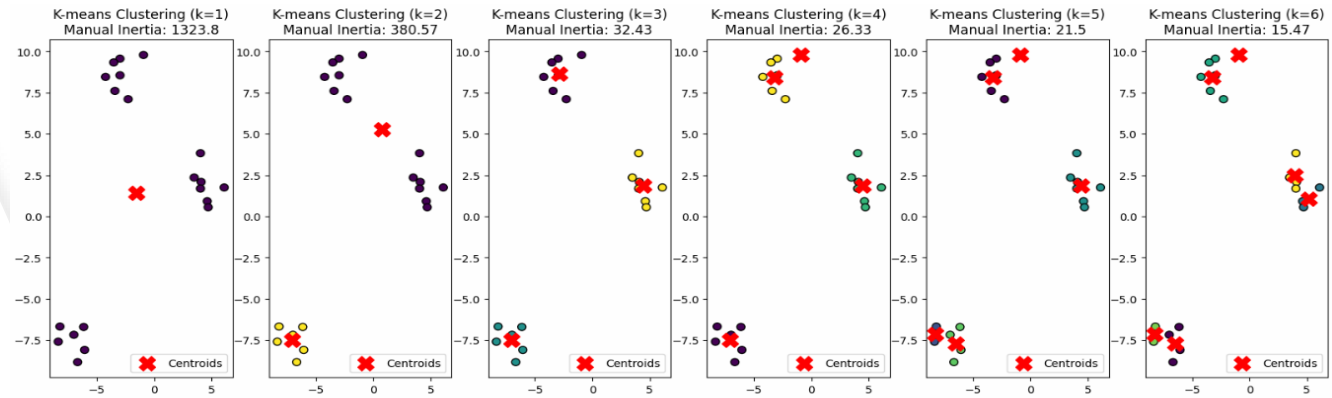


Silhouette Score: The silhouette score measures how similar an object is to its own cluster compared to other clusters. It ranges from -1 to 1, where a high value indicates that the object is well matched to its own cluster and poorly matched to neighboring clusters.



Visual Inspection: Visualization techniques like scatter plots, t-SNE, or PCA can help you visually inspect the quality of clustering. If the clusters are well-separated and distinct, it suggests a good unsupervised model.

Illustrate Inertia or Sum of Squared Distances (SSD):



1. Squared Distance:

1. For each data point in a cluster, the squared distance to its cluster's centroid is calculated. The squared distance is used to give more weight to points that are farther away, emphasizing their impact on the overall measure.

2. Summation:

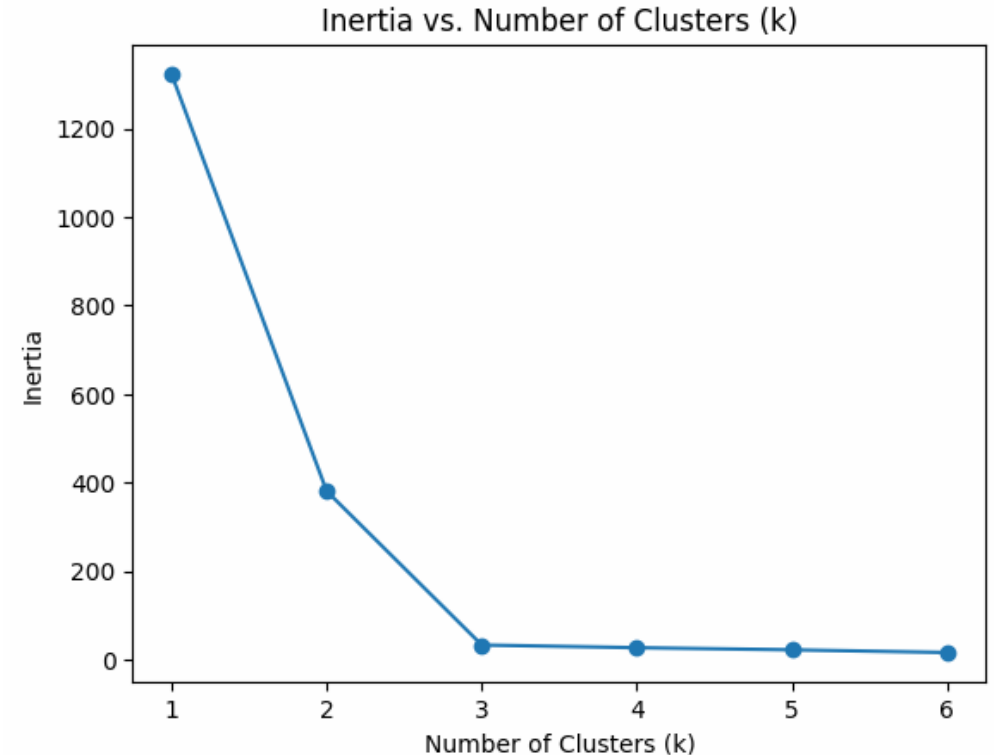
1. The squared distances for all data points within a cluster are summed. This results in a single value representing the overall dispersion of data points within that cluster.

3. Across All Clusters:

1. The SSD is calculated by summing the squared distances for all clusters. In the context of k-means clustering, this involves calculating the SSD for each cluster and then summing those values.

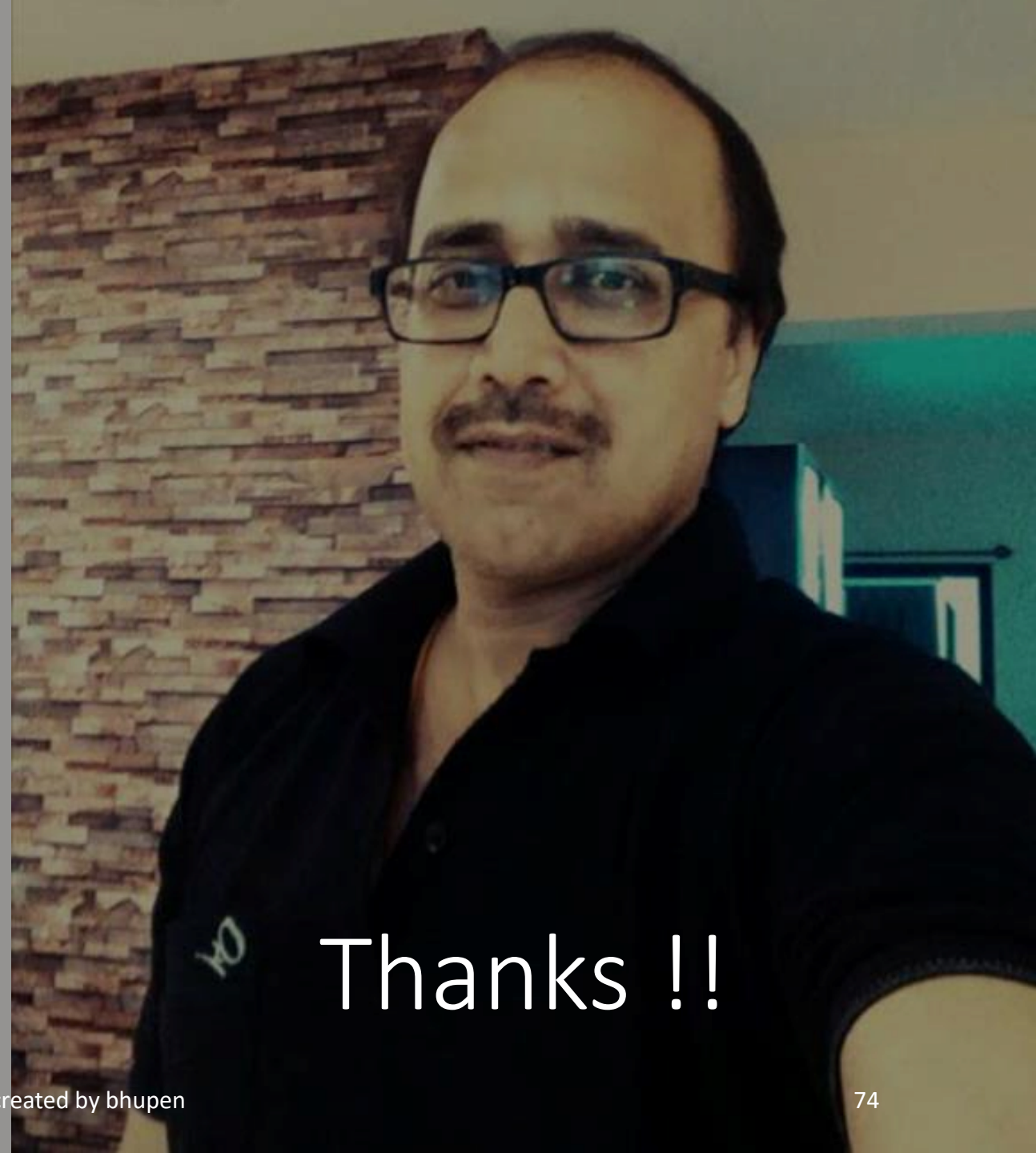
4. Objective Function:

1. In clustering algorithms like k-means, the goal is to minimize the SSD.



Types of Machine learning

Distance-Based Models:	<i>Example:</i> k-Nearest Neighbors (k-NN) <i>Description:</i> These models make predictions based on the distances between data points. k-NN, for instance, predicts the label of a data point based on the labels of its nearest neighbors in the feature space.
Probability-Based Models:	<i>Example:</i> Naive Bayes, Logistic Regression <i>Description:</i> These models are based on probability theory. Naive Bayes assumes independence between features, while logistic regression models the probability of an event occurring.
Tree-Based Models:	<i>Examples:</i> Decision Trees, Random Forests, Gradient Boosted Trees <i>Description:</i> These models use tree-like structures to make decisions. Decision trees split the data based on feature conditions, and ensemble methods like Random Forests combine multiple trees for improved performance.
Vector-Based Models:	<i>Examples:</i> Support Vector Machines (SVM) <i>Description:</i> SVMs, for instance, operate in a high-dimensional space and find a hyperplane that best separates different classes. They are effective for both classification and regression tasks.
Neural Network Models:	<i>Examples:</i> Feedforward Neural Networks, Convolutional Neural Networks (CNN), Recurrent Neural Networks (RNN) <i>Description:</i> Neural networks are composed of interconnected nodes (neurons) organized into layers. Feedforward networks process data in a sequential manner, while CNNs are effective for image-related tasks, and RNNs are suitable for sequence data.
Ensemble Models:	<i>Examples:</i> Random Forests, Gradient Boosted Trees, AdaBoost <i>Description:</i> Ensemble models combine the predictions of multiple base models to improve overall performance. Random Forests, for example, train multiple decision trees independently and average their predictions.



Thanks !!